

COMS W4156 Advanced Software Engineering (ASE)

September 17, 2024

On this day ...



We Start Recording Attendance TODAY

We will use zoom's built-in "usage reports", which show each attendee's zoom name, join time, leave time, and duration. To get full credit you need to attend for at least an hour (unless the class ends early). If you disconnect and reconnect, we can put the two (or more) together as long as it's the same zoom name

CVN students who cannot attend one or more zoom class sessions should do the "[CVN-only](#)" assignment for each session they miss



Non-CVN students can do the CVN-only assignment instead of attending a given class session **only** if they have an acceptable (to me) excuse and send it to me on edstem. Example acceptable: religious observance, illness or injury *sufficiently severe* that you cannot attend on zoom, job interview. Example unacceptable: Forgot, slept late, busy

Agenda

1. **RESTful APIs**
2. RESTful API demo (C++ & Java): Siddharth Ananth and Robin Liu
3. Team Project Coming Soon
4. Other Upcoming



Reprise: Libraries Run On The Host Machine

When you run software on your machine, then the libraries used in your software's code are compiled or bundled together with your software and also execute on your machine

Your software is a *client* of the library(ies)

Your software has its own copy of each library, and your software is (effectively) the only client of the copy

Something has to connect your code to the library code



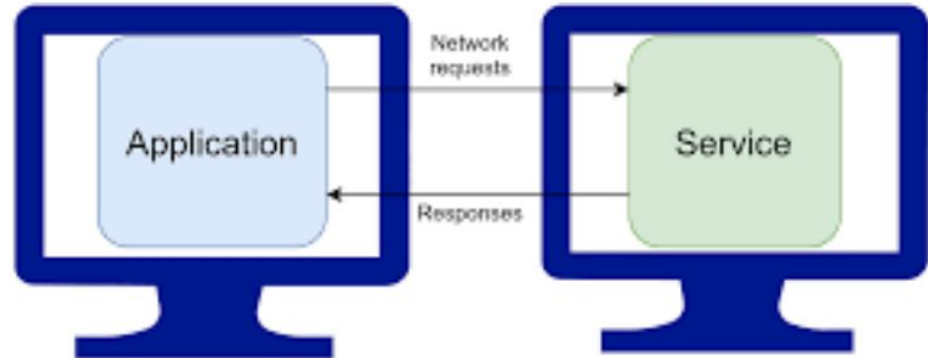
Reprise: Services Run Somewhere On The Network

When you run software on your machine, then the services used by your software's code can execute anywhere reachable from your machine

Your software is a *client* of the service(s)

Your software might share the service with other copies of your software and with independent software from other developers, teams, organizations, all of which are clients of the same service

Something has to connect your code, and all that other code, to the service's code



What Is That Something?

For a same-machine same-language library, the compiler and/or runtime environment handles everything

For a different-machine any-language service, Internet-based distributed computing infrastructure is needed (in addition to compiler(s) and/or runtime(s)). We do not care how Internet works

For the purposes of the miniproject, we only consider RESTful infrastructure

For the purposes of the team project, we will later consider RPC and some other kinds of API infrastructure



shutterstock.com · 263844431

RESTful Infrastructure

RESTful infrastructure ~=

World Wide Web

RESTful and REST mean the same thing in this context, we will use these terms interchangeably

early milestones	Key Layers of the Internet	milestones
email@-1971 Ray Tomlinson	CONTENT	1987-HyperCard Bill Atkinson
Archie-1990 Emtage & Deutsch	SEARCH ENGINE*	1998-Google Brin & Page
DOS Houdini-1986 Neil Larson	BROWSERS	1993-Mosaic Marc Andreessen
(Vannevar Bush, Ted Nelson, Douglas Engelbart)	WORLD WIDE WEB	1990-http:// Tim Berners-Lee
ARPANET-1969 J.C.R. Licklider	INTERNET	1975-TCP/IP Cerf & Kahn
SAGE-1956 George Valley	NETWORKS	1973-Ethernet Robert Metcalfe
Z3-1941 Konrad Zuse	COMPUTERS	1976-Apple Jobs & Wozniak

RESTful APIs

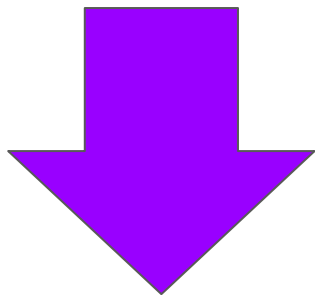
A RESTful service is usually accessed remotely via [HTTP](#) (HyperText Transfer Protocol) and/or [HTTPS](#) (Secure HTTP or HTTP with Security)

The RESTful service defines a set of [URLs](#) (Uniform Resource Locators) and CRUD operations ("[HTTP methods](#)") that constitute the service's API

HTTP Method to Action mapping

Method / Verb		Action	
HTTP	→	CRUD	Description
POST	→	CREATE	Create new object / resource
GET	→	READ	Retrieve resource details
PUT	→	UPDATE	Replace/Update Existing Resource
PATCH	→	Partial Update	Partial update to existing resource
DELETE	→	DELETE	Remove an existing resource

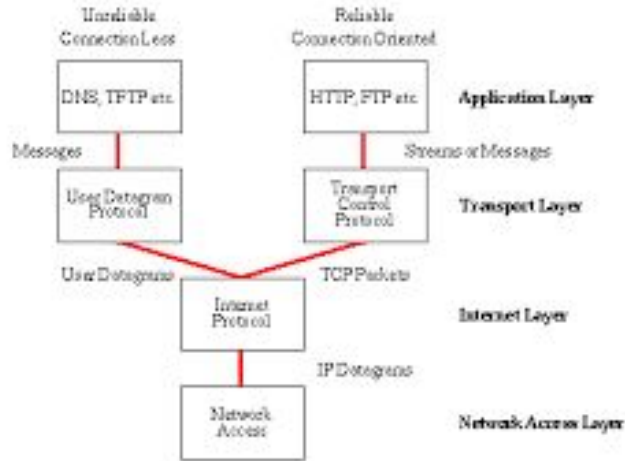
You Have All Used HTTP(S) with RESTful APIs



HTTPS is the *protocol*, www.columbia.edu is the *URL*, and <https://www.columbia.edu> (with default GET method) is an API call to a "service" that returns content to your browser (plus an unshown [status code](#), 200=OK)



Protocols vs. APIs



Protocol = the rules for initiating, continuing and completing communications among software components and how the communication messages are formatted in terms of bytes (e.g., all of you have also used [SMTP](#) protocol)

API (Application Programming Interface) = the set of function calls or other entry points that a given software component implements and its clients use

We are concerned in this class primarily with APIs, but you need to use some protocol under the covers to send API calls between machines

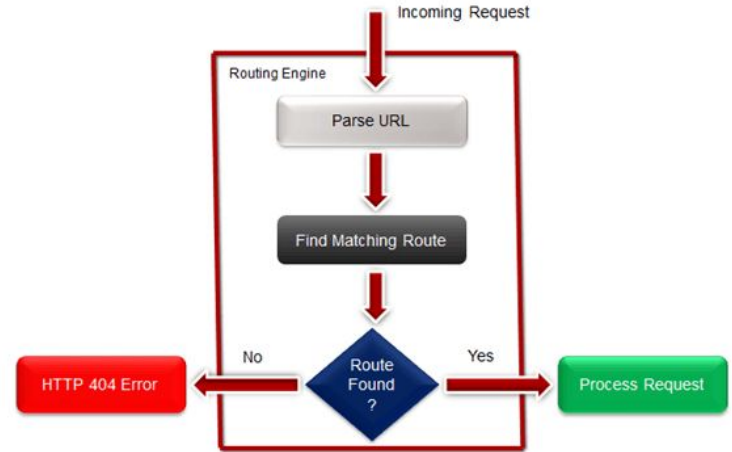
Routes and Endpoints

In RESTful terminology, a URL is often called a “[route](#)”

The domain in `http://mydomain.com/file/system/path...` tells the client (and any intermediate internet nodes) how to get to the machine where the service runs

The path looks like a file system path and can indeed be interpreted as a file system path by the recipient service - or any other way whereby it finds the function to call

HOW ROUTING WORKS

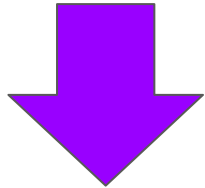


The “[endpoint](#)” is after the domain and path, telling the service which function is being called, `http://example.com/very/long/path/to/where/is/the/function/endpoint`

The main thing this API call does is fetch predefined content, but it also calls some javascript code that fetches and/or constructs additional content (there's also some analytics computation)

Returning this content to the caller (a browser in this case) is referred to as "REpresentational State Transfer" = REST

This particular content is formatted as an HTML document, but when RESTful APIs are used for distributed computing, the returned content is typically a JSON or XML document



Here is some [XML](#)

<|As>

<|A>

<firstName>Griffin</firstName>

<lastName>Newbold</lastName>

<photo>...</photo>

<homepage>...</homepage>

<uni>gcn2106</uni>

...

</|A>

<|A>

...

</|A>

...

</|As>

JSON vs XML

	JSON	XML
Readability	Very easy to read as its based on defining objects and values	Slightly less easy to read as data is contained within markup tags
Compact Code	Less code is created than XML	Requires more code than JSON
Parsing Speed	Quicker than XML and data us clearly defined as object and value	Slower than JSON as the data has to be extracted from the tags
Ease of Creation	Easier to create as the syntax of the coding is easier	Slightly more syntax to learn than JSON
Flexibility & Extendability	Works with a limited range of data types which may not be sufficient for all applications	Similar to programming so therefore more knowledge is required
Security	JSON is a subset of JavaScript and for that reason it can be used to run malicious code.	XML is more secure that JSON for the reason stated.

Here is some [JSON](#)

{"|As":[

{ "firstName":"Griffin", "lastName":"Newbold",
"photo": ..., "homepage":...,
"uni":"gcn2106", ...

},

{ ... },

...

}]

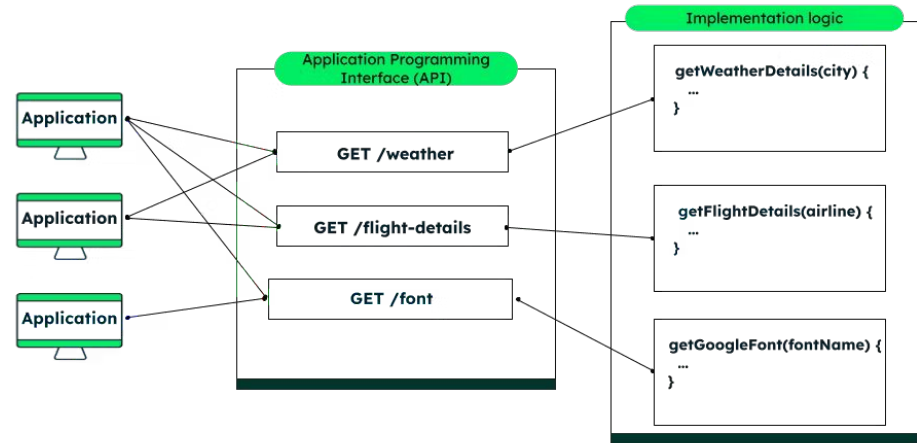
Similar HTML content is [here](#)

Do RESTful APIs Have Parameters?

Some APIs have no parameters

<https://www.columbia.edu> has no explicit parameters, but there's an implicit time/date parameter - the caller wants to receive the content as it is now, not as it was yesterday, last week or last year

Most service APIs (and library APIs) have parameters, just like most other non-trivial functions in other code



How To Provide Parameters To RESTful API Calls

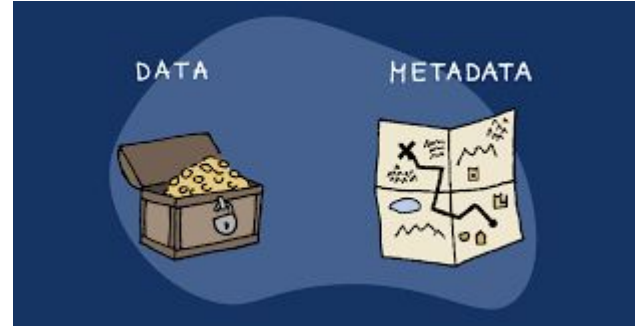
Path parameters:

`http://example.com/path/to/function/{input}`

`http://example.com/path/to/function/{input}/morepath`

Query parameters:

`http://example.com/path/to/function?param1=input¶m2=anotherinput`



The parameters influence what the service does and what data it returns, just like the parameters to function calls in apps, but GET calls should not have side-effects

Except might update [metadata](#) like a counter of how many times a resource has been accessed or the timestamp of the most recent access

There Is More To HTTP Than URLs

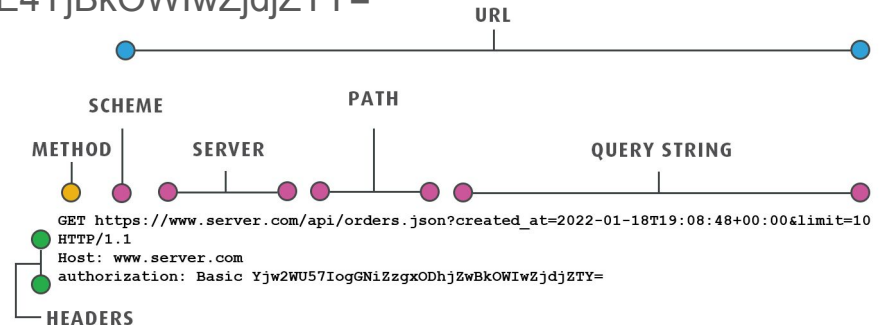
HTTP *headers* and *bodies* let the client and the server pass additional information with an HTTP request or response. Headers typically are used to pass authentication values, indicate the content type of a request, the desired response content type, etc.

An HTTP header consists of its case-insensitive name followed by a colon (:), then by its value. Whitespace before the value is ignored.

User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; WOW64; Trident/5.0)

Authorization: Basic YjhiZWU5ZGNIYzgxODhjNIZjE4YjBkOWlwZjdjZTY=

Accept : application/json



RESTful APIs Can Do More Than GET

HTTP bodies are not normally used with the GET method

But if RESTful APIs were limited to the HTTP GET method (READ), they would not be very useful for distributed computing

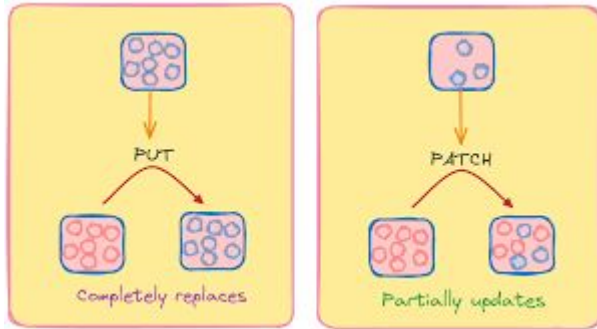
Operation	SQL	HTTP	DDS
Create	INSERT	PUT / POST	write
Read (Retrieve)	SELECT	GET	read / take
Update (Modify)	UPDATE	PUT / PATCH	write
Delete (Destroy)	DELETE	DELETE	dispose

Structured Query Language ([SQL](#)) is standard relational database query language. This is what they teach in [4111](#) and might be useful for some team projects for this class
Data Distribution Service ([DDS](#)) is a publish-subscribe middleware standard widely used in industrial control, factory automation, IoT and others, outside scope of class

HTTP Methods PUT, POST and PATCH

Services typically maintain persistent state that can be modified (CREATE, UPDATE, DELETE) as well as accessed (READ) by clients making calls to the service's API

Parameters to PUT, POST, PATCH – intended to influence these side-effects – are provided in the HTTP body not the header or URL



```
POST - https://www.server.com/admin/api/order
HTTP/1.1
Host: www.server.com
authorization: Basic YjhiZWU5ZGNIYzgxODhjZGBkOWIwZjdjZTY=
Body - {
  "reference": "PO16262999",
  "customer" : "1200",
  "lineItems": [
    {
      "lineId": 23894,
      "sku": "A1-103/0",
      "qty": 2,
      "description" : "A desk lamp"
    },
    {
      "lineId": 4759,
      "sku": "A1-401/0",
      "qty": 1,
      "description" : "Some Bulbs"
    }
  ]
}
```

BODY

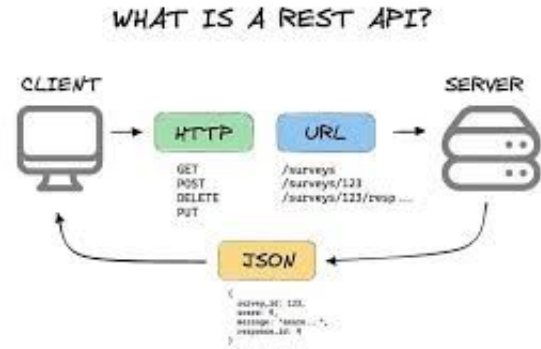
Agenda

1. RESTful APIs
2. RESTful API demo (C++ & Java): Siddharth Ananth and Robin Liu
3. Team Project Coming Soon
4. Other Upcoming



Siddharth Ananth and Robin Liu RESTful API demos (C++ & Java)

[demo slides](#)



Agenda

1. RESTful APIs
2. RESTful API demo (C++ & Java): Siddharth Ananth and Robin Liu
3. Team Project Coming Soon
4. Other Upcoming



Team Project Milestones

Team Formation due this Friday, September 20, 11:59pm

Team Project Proposal due one week later September 27

Ideally 4 students per team, 5 students ok. Make sure:

- Everyone on the team submits the same thing
- Everyone fluent in team's choice of Java or C++
- Everyone has same or compatible development platform
- Everyone has compatible schedules

We will spend a little time at the beginning of class Thursday on last-minute team formation



Your Team Should Propose To Build A Service

In theory, your service should be usable by lots of different clients developed by arbitrary developers, teams, organizations that you know nothing about

Your service must be hosted in the cloud so those clients can use it (we will provide GCP credits)

In practice, your own team will build a sample client as part of the second iteration. The sample client can run on localhost



Your Project Must Target an "Acceptable User Group"

- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Maybe others with societal benefit - ask me *in advance*



Agenda

1. RESTful APIs
2. RESTful API demos (C++ & Java): Siddharth Ananth and Robin Liu
3. Team Project Coming Soon
4. Other Upcoming



Remaining Miniproject Deadlines

[Part Individual Miniproject part 2](#) due this Friday September 20 (students who registered for class September 9 or later should submit [part 1](#) together with part 2)

[Part Individual Miniproject part 3](#) due the following Friday September 27

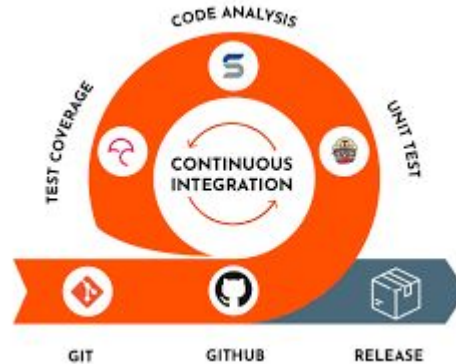


Thursday

Finish anything we didn't get to today

Continuous Integration

Github Actions CI demo





Ask Me Anything